使いこなせると有利な機能

使いこなせると有利な機能

Math関数

Math関数は数学的な処理をしてくれる関数です。
Using Systemの中に入っています。

Linq関数

Linq関数は配列に特定の操作をする事が出来る関数です。。
Using Linqの中に入っています。

Math関数でよく使う物

Math.Abs(数値)	絶対値
Math.Floor(数値)	少数切り捨て
Math.Max(数値、数値)	大きい方を返す
Math.Min(数値、数値)	小さい方を返す
Math.Pow(数値、数値	累乗
Math.Sing(数値)	符号を返す

Linq関数とは

Linq関数は配列に特定の操作をする事が出来る関数です。。

例えば、前回の授業で出てきた下記の文は
配列すべての文字を数値に変える処理でした。

```
string line = Console.ReadLine();
int[] numbers = line.Split(" ").Select(int.Parse).ToArray();
```

数値の配列に対してLinq関数で出来る事

.Sum()	配列すべての合計
.Average()	配列内の数値の平均
.Max()	配列内の中で最も大きい数
.Min()	配列内の中で最も小さい数
.Count(関数)	関数の結果に合致する数
.Where(関数)	配列から関数の結果に合致するもの以外削除
.Select(関数）	配列の数に対して関数の変更を加える
.OrderBy(関数）	照準ソート
.ToArray()	Linq結果を配列にする
.ToList()	Linq結果をリストにする

関数を入れる変数

```
0 個の参照
static void Main(string[] args)
{
    Action spaceKey = Move;
    Action enterKey = Stop;

    bool isSpaceKey = false;
    bool isEnterKey = false;

    //  スペースキーが押されたらスペースキーの関数を実行する
    if(isSpaceKey)
    {
        spaceKey.Invoke();
    }

    //  エンターキーが押されたらエンターキーの関数を実行する
    if(isEnterKey)
    {
        enterKey.Invoke();
    }
}

1 個の参照
private static void Move()
{
    Console.WriteLine("動くよ");
}

1 個の参照
private static void Stop()
{
    Console.WriteLine("止まるよ");
}
```

変数には関数を入れる変数があります。

使う例としては、
・配列やリストに一括で処理をする
・キー操作の登録をする
・ボタンなどに登録する

こういった時に関数を保存する変数の
Action　アクション
Func　ファンクションを使います。
※Action は戻り値無しの時
　Func　は戻り値がある時

.Count(関数)を見てみよう

```
0 個の参照
static void Main(string[] args)
{
    int[] numbers = { 1, 2, 3, 4, 5, 6, 7, 8, 9, };
    int over5Count = numbers.Count(IsOver5);

    /* over5Countは4になる */

    //下記のように一旦変数にも入れられる
    //Func<int, bool> func = IsOver5;
    //int over5Count = numbers.Count(func);
}

1 個の参照
private static bool IsOver5(int value)
{
    if(value > 5)
    {
        return true;
    }
    else
    {
        return false;
    }
}
```

こんな形でまずはカウントする条件の関数を作り、
その関数を呼び出さず、
そのままCountの引数に入れる事で
あとは勝手にその関数を使って
数を数えてくれます。

処理のイメージ

```
0 個の参照
static void Main(string[] args)
{
    int[] numbers = { 1, 2, 3, 4, 5, 6, 7, 8, 9, };
    int over5Count = numbers.Count(IsOver5);
}
```

＝

```
static void Main(string[] args)
{
    int[] numbers = { 1, 2, 3, 4, 5, 6, 7, 8, 9, };

    int count = 0;
    for (int i = 0; i < numbers.Length; i++)
    {
        if (IsOver5(numbers[i]))
        {
            count++;
        }
    }
}
```

このように、Linqに関数を渡すと
for文で１個１個その関数を
通過させてくれるイメージです。

Action・Funcの注意点

```
0 個の参照
static void Main(string[] args)
{
    int[] numbers = { 1, 2, 3, 4, 5, 6, 7, 8, 9, };

    Func<int, bool> func = IsOver5;
    Func func2 = IsOver5;

    func.Invoke(0);
    func.Invoke();
}
```

同じFucn変数でも、引数や戻り値で別の形になります。
なので、画像のように引数と戻り値の形を指定しないとエラーが出てしまいます。

また、Invoke()で呼び出す際には、その引数の形に沿った、値を入れてあげる必要があります。

アクションをマジックナンバーのように記入する

```
int[] numbers = { 1, 2, 3, 4, 5, 6, 7, 8, 9, };

Func<int, bool> func = IsOver5;
Func<int, bool> func2 =
    value =>
    {
        if (value > 5)
        {
            return true;
        }
        else
        {
            return false;
        }
    };
```

さっきのIsOver５のように関数の引数として１回しか使用しないのにわざわざ関数書くのが面倒くさい場合にマジックナンバーのように直接記入する事が出来ます。

これをラムダ式といいます。

ラムダ式の書き方

```
int[] numbers = { 1, 2, 3, 4, 5, 6, 7, 8, 9, };

Func<int, bool> func = IsOver5;
Func<int, bool> func2 =
    value =>
    {
        if (value > 5)
        {
            return true;
        }
        else
        {
            return false;
        }
    };
```

ラムダ式は下記のように書きます。

```
引数名　=>
{
　処理
};
```

ラムダ式の書き方　略式

```
Func<int, bool> func = IsOver5;

Func<int, bool> func2 =
    value =>
    {
        if (value > 5)
        {
            return true;
        }
        else
        {
            return false;
        }
    };

Func<int, bool> func3 = value => value > 5;
```

※上のfunc３つは全部同じ

ラムダ式は処理が１行だけの場合
省略した書き方が出来ます。

＜本来＞
引数名　=>
{
　処理
};

＜省略＞
引数名　=> 処理;
この時戻り値がある場合は
return を省略します。

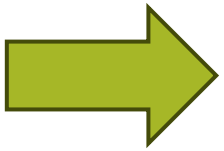
ラムダ式の書き方　略式

Linqとラムダ式を活用すると
プログラムが遥かにスマートになります。

```
0 個の参照
static void Main(string[] args)
{
    int[] numbers = { 1, 2, 3, 4, 5, 6, 7, 8, 9, };

    int count = 0;
    for (int i = 0; i < numbers.Length; i++)
    {
        if (IsOver5(numbers[i]))
        {
            count++;
        }
    }
}

1 個の参照
private static bool IsOver5(int value)
{
    if(value > 5)
    {
        return true;
    }
    else
    {
        return false;
    }
}
```



```
0 個の参照
static void Main(string[] args)
{
    int[] numbers = { 1, 2, 3, 4, 5, 6, 7, 8, 9, };
    int count = numbers.Count(vaule => vaule > 5);
}
```

練習問題

荷物検査　C#編
背の順　C#編
宿題管理　C#編
高い寿司を食いたい　C#編
５以上の整数の合計　C#編
地下アイドルの夢　C#編